Ruta Fantasma

Memoria del proyecto Scrum — Desarrollo de Interfaces

Product Owner: Daniel Martín Scrum Master: Aitor
Desarrollo: Asier García Catalán, Daniel Rodríguez, Adam

1º DAM — Curso 2026-2027

Índice

1 Introducción **1**

2 El producto **2**
2.1 Visión . . . 2
2.2 Usuarios . . . 2
2.3 Product Backlog inicial . . . 2

3 Equipo y forma de trabajo **2**
3.1 Roles . . . 2
3.2 Tecnologías (stack) . . . 2
3.3 Herramientas . . . 3
3.4 Ceremonias . . . 3

4 Sprint 1 **3**
4.1 Sprint Planning . . . 3
4.2 Trabajo realizado . . . 3
4.3 Configuración del repositorio y del flujo de trabajo . . . 3
4.4 DT2 · Preparación y publicación: proyecto Angular con zonas y PWA . . . 4
4.5 Organización del árbol del proyecto . . . 6
4.6 PS-24 · API para crear y listar rutas . . . 6
4.7 PS-19 · API de registro e inicio de sesión . . . 7
4.8 PS-25 · Colección de Postman . . . 9
4.9 PS-18 · Entidad y tabla usuario en MySQL . . . 10
4.10 PS-23 · Tablas ruta, carrera, punto y sector . . . 11
4.11 Carpeta de diseño para los mockups . . . 12
4.12 PS-26 · Pantalla con la lista de rutas . . . 13
4.13 PS-20 · Pantallas de registro e inicio de sesión . . . 14
4.14 PS-21 · Servicio HTTP base de Angular . . . 16
4.15 Flujo de trabajo con ramas y Pull Requests . . . 17

5 Sprint 2 **18**
5.1 Sprint Planning . . . 18
5.2 Trabajo realizado . . . 19
5.3 PS-32 · Filtro de GPS: precisión y saltos imposibles . . . 19
5.4 PS-31 · Servicio de cálculo: fórmula de Haversine y distancia . . . 20

6 Conclusiones **21**

1 Introducción

Esta memoria recoge el desarrollo del proyecto **Ruta Fantasma**, realizado en la asignatura de Desarrollo de Interfaces siguiendo la metodología Scrum.

(Completad aquí: objetivo del trabajo, motivación de la idea y qué se espera conseguir al final.)

2 El producto

2.1 Visión

Ruta Fantasma es una aplicación de reloj para correr contra tu mejor marca. En pantalla aparecen dos puntos: tú y tu “sombra” (tu mejor carrera en ese recorrido). Durante la carrera, la app avisa por sectores en verde (vas mejor) o en rojo (vas peor), como los tiempos por sector de la Fórmula 1.

2.2 Usuarios

(Quién va a usar la app y qué necesita.)

2.3 Product Backlog inicial

ID	Historia de usuario	Prioridad	Estimación
HU-01	Como corredor, quiero ver mi posición y la de mi sombra para saber si voy mejor o peor	Alta	*(pts)*
HU-02	Como corredor, quiero recibir avisos verdes/rojos por sector para ajustar el ritmo	Alta	*(pts)*
HU-03	*(…)*		

3 Equipo y forma de trabajo

3.1 Roles

Rol	Persona	Responsabilidad
Product Owner	Daniel Martín	Define y prioriza el Product Backlog
Scrum Master	Aitor	Facilita las ceremonias y elimina impedimentos
Desarrollador	Asier García Catalán	Desarrollo
Desarrollador	Daniel Rodríguez	Desarrollo
Desarrollador	Adam	Desarrollo

3.2 Tecnologías (stack)

Parte	Tecnología	Motivo
Frontend	Angular + TypeScript	Vista en clase; permite simular la pantalla del reloj en el navegador
Backend	Java + Spring Boot	Visto en clase; organiza la API REST por capas (controlador, servicio, repositorio)
Base de datos	MySQL (con JPA/Hibernate)	Los datos son relacionales (usuarios, rutas, puntos, sectores y carreras) y MySQL es el gestor visto en clase

3.3 Herramientas

- **GitHub:** repositorio común del código y de esta memoria.
- **GitHub Issues y Projects:** Product Backlog y tablero del sprint (Por hacer / En curso / Hecho).
- **Claude Code:** asistente de programación que usa cada desarrollador. Todos comparten el mismo contexto mediante el archivo `CLAUDE.md` del repositorio.
- **Generación automática de la memoria:** cada avance se documenta en Markdown y GitHub Actions genera este PDF en cada subida.

3.4 Ceremonias

- **Sprint Planning:** al inicio de cada sprint.
- **Daily:** *(frecuencia real del equipo)*.
- **Sprint Review y Retrospectiva:** al final de cada sprint; se resumen en el apartado de cada sprint.

4 Sprint 1

Fechas: *(inicio — fin)*

Objetivo del sprint: *(una frase)*

4.1 Sprint Planning

Historia / tarea	Responsable	Estimación
(HU-01 …)	*(nombre)*	*(pts)*

4.2 Trabajo realizado

4.3 Configuración del repositorio y del flujo de trabajo

Autor: Asier García · **Fecha:** 13/10/2026 · **Issue:** —

4.3.1 Qué se ha hecho

Se ha preparado el repositorio común para que los tres desarrolladores trabajen con el mismo contexto y para que la memoria se genere sola.

4.3.2 Cómo se ha hecho

- Se creó el archivo `CLAUDE.md` con el contexto del proyecto y las reglas del equipo. El asistente de cada desarrollador lo lee al empezar, de modo que los tres trabajan con la misma información.
- Se añadieron dos comandos, `/empezar` y `/terminar`, que automatizan actualizar el repositorio al empezar y documentar, hacer commit y subir el trabajo al terminar.
- La memoria se escribe en Markdown dentro de `docs/memoria/`. Cada tarea terminada genera un archivo propio, lo que evita conflictos entre compañeros.
- Un flujo de GitHub Actions convierte todos esos archivos en `docs/memoria.pdf` cada vez que se sube un cambio a la rama principal.

4.3.3 Por qué

Reduce el tiempo dedicado a tareas repetitivas (subir cambios, maquetar la memoria) y garantiza que la documentación esté siempre al día con el código.

4.3.4 Problemas encontrados

Ninguno por ahora.

4.4 DT2 · Preparación y publicación: proyecto Angular con zonas y PWA

Autor: Asier García · **Fecha:** 14/10/2026 · **Issue:** DT2

4.4.1 Qué se ha hecho

Se ha creado la base del frontend de Ruta Fantasma: un proyecto Angular dividido en tres zonas independientes (`/reloj`, `/app` y `/web`) y configurado como aplicación web progresiva (PWA), de modo que se puede instalar en el móvil y funciona sin conexión. Cada zona tiene ya una primera pantalla de ejemplo sobre la que el resto del equipo puede seguir trabajando.

4.4.2 Cómo se ha hecho

- **Creación del proyecto.** Con Angular CLI 22 se generó el proyecto en `src/frontend`, con enrutado, CSS y sin renderizado en servidor (`ng new ruta-fantasma --routing --style=css --ssr=false`).
- **Zonas con carga diferida.** En `src/app/app.routes.ts` se definieron las tres zonas mediante `loadChildren`. Cada una vive en su propia carpeta `src/app/zonas/<zona>/` con su archivo de rutas (`reloj.routes.ts`, `app.routes.ts`, `web.routes.ts`) y un componente de inicio. La raíz `/` y cualquier dirección desconocida redirigen a `/web`.

  - **`/web`:** página pública de presentación del producto, con titular y tres tarjetas de características.
  - **`/app`:** diseño de móvil con la lista “Mis rutas” (vacía de momento) y un menú inferior.
  - **`/reloj`:** esfera redonda que simula la pantalla del reloj, con los dos puntos (corredor y sombra), el tiempo y la diferencia del sector. El borde cambia a verde o rojo según se vaya mejor o peor. Usa *signals* de Angular y datos de ejemplo hasta que se conecte el GPS.
- **Estilos comunes.** En `src/styles.css` se definieron variables CSS (colores de fondo, morado “fantasma”, verde y rojo de sector) para que las tres zonas tengan una imagen coherente.
- **PWA.** Con `ng add @angular/pwa` se añadieron el *service worker* (`ngsw-config.json`), el manifiesto y los iconos. Después se personalizó el manifiesto (nombre “Ruta Fantasma”, idioma español, colores del tema) y el `index.html` (idioma, título, descripción, `theme-color` e icono para iOS).
- **Pruebas.**
  - **Pruebas unitarias:** se escribieron con Vitest para comprobar que cada ruta carga su zona, que las redirecciones funcionan y que el reloj formatea bien el tiempo (`12:34`) y la diferencia (`-2,4 s`). Pasan las 10.
  - **Pruebas en el navegador:** se comprobaron las tres zonas en tamaño de escritorio y de móvil. Además, sobre la versión de producción se verificó que el *service worker* se instala, guarda la aplicación en caché y que `/reloj` se abre con el servidor apagado.

4.4.3 Por qué

- **Tres zonas separadas:** cada desarrollador puede trabajar en una zona sin tocar los archivos de los demás, lo que reduce los conflictos en Git. Además, con la carga diferida el navegador solo descarga el código de la zona que se está usando. La compilación genera un fichero distinto por zona, de 1 a 3 kB cada uno.
- **PWA:** Ruta Fantasma se usa corriendo por la calle, donde la conexión puede fallar. Con el *service worker* la aplicación sigue funcionando sin red y puede instalarse como si fuera una app nativa, sin tener que publicarla en una tienda de aplicaciones.
- **Angular 22 con componentes *standalone* y *signals*:** es la forma recomendada actualmente por Angular y simplifica el código.

4.4.4 Problemas encontrados

- Los tests generados por defecto fallaban porque los componentes usan `routerLink` y el test no tenía router. Se resolvió añadiendo `provideRouter` en cada test.
- En las pruebas de rutas se comprobaba mal qué componente aparecía en pantalla. Se cambió a `RouterTestingHarness.navigateByUrl(url, Componente)`, que verifica directamente que se muestra el componente esperado.
- El *service worker* no se activa con `npm start` (modo desarrollo), solo en la versión de producción (`ng build`). Es el comportamiento normal de Angular y queda anotado en `CLAUDE.md`.

4.5 Organización del árbol del proyecto

Autor: Asier García · **Fecha:** 20/10/2026 · **Issue:** —

4.5.1 Qué se ha hecho

Se ha definido la estructura de carpetas completa del repositorio, tanto del frontend como del backend, para que cada parte del código tenga un lugar fijo antes de empezar a desarrollar las historias de usuario.

4.5.2 Cómo se ha hecho

- En el frontend (`src/frontend/src/app/`) se añadieron, junto a las `zonas/` ya existentes, dos carpetas nuevas: `core/` (servicios y modelos de datos que se usan en toda la aplicación) y `compartido/` (componentes y utilidades reutilizables entre zonas).
- Se creó el esqueleto del backend en `src/backend/`, organizado por capas: `api/` (endpoints), `controladores/`, `servicios/` (lógica de negocio), `modelos/` (MongoDB), además de `config/`, `middlewares/`, `utilidades/` y `tests/`.
- Se añadió un `.env.example` con las variables necesarias y un `README.md` del backend explicando cada carpeta.
- Se actualizó el árbol de `CLAUDE.md` para que los asistentes de todo el equipo conozcan la nueva estructura.

4.5.3 Por qué

Con una estructura acordada desde el principio, los desarrolladores saben dónde colocar cada archivo y se reducen los conflictos al trabajar en paralelo. La separación por capas del backend facilita además probar y modificar cada parte por separado.

4.5.4 Problemas encontrados

Ninguno.

4.6 PS-24 · API para crear y listar rutas

Autor: Adam (integrado en el repositorio por Asier García) · **Fecha:** 21/10/2026 · **Issue:** PS-24

4.6.1 Qué se ha hecho

Se han creado los endpoints `POST /api/rutas` (crear una ruta), `GET /api/rutas` (listar las rutas del usuario) y `GET /api/rutas/{id}` (ver una ruta). Al crear una ruta se guardan sus puntos GPS en MySQL y se divide automáticamente en sectores. Solo pueden usarlos usuarios que hayan iniciado sesión, y cada uno ve únicamente sus rutas.

4.6.2 Cómo se ha hecho

- **Protección con el token de PS-19.** Un `HandlerInterceptor` (`TokenInterceptor`) se ejecuta antes del controlador en `/api/rutas/**`: lee la cabecera `Authorization: Bearer <token>`, comprueba la firma y la caducidad con

TokenService y guarda el id del usuario en la petición. Si falta o no es válido responde 401 { error }. El controlador recibe el id con @RequestAttribute.
- **DTO separados de las entidades** (RutaRequest, RutaResponse, PuntoDto, SectorDto), para no exponer la estructura interna de la base de datos y controlar exactamente el JSON. La respuesta usa los mismos nombres de campo que el modelo Ruta del frontend.
- **Validaciones:** nombre obligatorio (máximo 60 caracteres), al menos 2 puntos, latitud entre -90 y 90, longitud entre -180 y 180, tiempo no negativo y entre 1 y 10 sectores. Se usa @Valid dentro de la lista para validar cada punto. Además, el servicio comprueba que los tiempos de los puntos no vayan hacia atrás.
- **División en sectores.** Si no se indica numeroSectores se usan 3, como en la Fórmula 1. Los puntos se reparten en tramos de tamaño parecido (con 7 puntos: 0→2, 2→4 y 4→6) y nunca se crean más sectores que tramos.
- **Guardado en cascada.** Se construye la Ruta con sus Punto y Sector y un solo save() lo inserta todo dentro de una transacción (@Transactional).
- **Privacidad:** si un usuario pide una ruta de otro, recibe 404, igual que si no existiera.
- **Pruebas** con MockMvc (7 nuevas, 21 en total): creación con sectores, límite de sectores, listado por usuario, ruta de otro usuario, peticiones sin token o con token falso, validaciones y tiempos incorrectos.

4.6.3 Por qué

- **Interceptor en lugar de Spring Security completo:** para dos endpoints es mucho más sencillo de entender y explicar, y cumple la misma función.
- **Cada usuario solo ve sus rutas:** son sus carreras personales; además así se protege su ubicación.
- **Calcular los sectores en el servidor:** el frontend no tiene que preocuparse de ello y todas las rutas se dividen igual.
- **IDs como texto en el JSON:** el frontend ya trabaja con id: string, y así no hay que cambiarlo.

4.6.4 Problemas encontrados

- Las pruebas de rutas y las de autenticación comparten la misma base de datos en memoria. Al borrar usuarios con rutas fallaba la clave foránea, así que las pruebas de rutas borran primero las rutas y limpian también al terminar (@AfterEach).
- Al integrar la tarea en el repositorio se añadieron RutaNoEncontradaException (404) y DatosRutaIncorrectosException (400) y sus manejadores en GlobalExceptionHandler, según lo que comprueban las pruebas.

4.7 PS-19 · API de registro e inicio de sesión

Autor: Adam (integrado en el repositorio por Asier García) · **Fecha:** 21/10/2026 · **Issue:** PS-19

4.7.1 Qué se ha hecho

Se han creado en el backend los endpoints POST /api/auth/registro y POST /api/auth/login. Al registrarse se guarda el usuario en la tabla usuario de MySQL

(PS-18) con la contraseña cifrada, y en los dos casos la API devuelve un token JWT y los datos públicos del usuario. Sigue el contrato que ya usa el frontend Angular (PS-20).

4.7.2 Cómo se ha hecho

- **Arquitectura por capas:** `AuthController` recibe la petición, `AuthService` contiene la lógica y `UsuarioRepository` accede a MySQL.
- **DTO con validaciones** (`RegistroRequest`, `LoginRequest`) usando Bean Validation: nombre obligatorio de hasta 40 caracteres, email con formato válido y contraseña de 6 a 72 caracteres. Con `@Valid`, Spring devuelve un `400` con un mensaje por campo.
- **Contraseñas con BCrypt** (`spring-security-crypto`). Solo se usa el cifrador, no todo Spring Security, para mantener el proyecto sencillo. En el login se compara con `matches()`, que no necesita descifrar nada.
- **Token JWT** con la librería JJWT (`TokenService`). El token lleva el id del usuario, el email y la fecha de caducidad (24 horas), y va firmado con una clave secreta que se lee de la variable `JWT_SECRETO`.
- **Emails normalizados** (sin espacios y en minúsculas), así `Ana@Correo.com` y `ana@correo.com` son la misma cuenta.
- **Errores** centralizados en `GlobalExceptionHandler`: `409 { error }` si el email ya existe, `401 { error }` si el login falla y `400 { campo: mensaje }` en las validaciones. Es el formato que traduce el interceptor de errores del frontend.
- **CORS** actualizado para permitir la cabecera `Authorization` desde `localhost:4200`.
- **Pruebas** con `@SpringBootTest` y `MockMvc` (6 nuevas, 9 en total): registro correcto (y que en la base de datos se guarda un hash BCrypt), email repetido, validaciones, login correcto, contraseña incorrecta y email inexistente.

4.7.3 Por qué

- **JWT en lugar de sesiones en el servidor:** el servidor no tiene que recordar quién ha iniciado sesión; basta con comprobar la firma del token. Encaja con una API REST y con el frontend, que ya envía `Authorization: Bearer <token>`.
- **El mismo error para email inexistente y contraseña incorrecta:** así nadie puede averiguar qué emails están registrados probando en el login.
- **BCrypt:** es lento a propósito y añade una sal aleatoria, de modo que dos contraseñas iguales generan hashes distintos.
- **Clave secreta en variable de entorno:** no se sube ninguna clave real al repositorio.

4.7.4 Problemas encontrados

- JJWT exige una clave de al menos 32 bytes para firmar con HS256. Se documentó en `application.properties` y se puso un valor de desarrollo suficientemente largo.
- Al integrar la tarea en el repositorio se añadieron las clases `EmailYaRegistradoException` y `CredencialesIncorrectasException`, con los mensajes que comprueban las pruebas.

4.8 PS-25 · Colección de Postman

Autor: Adam (integrado en el repositorio por Asier García) · **Fecha:** 21/10/2026 · **Issue:** PS-25

4.8.1 Qué se ha hecho

Se ha creado una colección de Postman, "Ruta Fantasma API", con las 11 peticiones necesarias para probar la API del backend: registro e inicio de sesión (PS-19) y creación y listado de rutas (PS-24). Cada petición incluye pruebas automáticas, así que con un solo clic (*Run collection*) se comprueba que toda la API funciona.

4.8.2 Cómo se ha hecho

- **Dos carpetas:** *Autenticación* (5 peticiones) y *Rutas* (6 peticiones). Para cada endpoint hay un caso correcto y los casos de error principales (400, 401, 404 y 409).
- **Variables de colección:** `baseUrl`, `email`, `password`, `token` y `rutaId`. Así, si cambia la dirección del servidor, solo hay que modificarla en un sitio.
- **Encadenamiento de peticiones con scripts:**
  - El *pre-request script* del registro genera un email nuevo cada vez (`ana<marca de tiempo>@correo.com`), para poder repetir la colección sin el error "email repetido".
  - Los scripts de prueba del registro y del login guardan el `token` de la respuesta, y el de "Crear ruta" guarda el `rutaId`, que usa después "Ver una ruta".
- **Autorización heredada:** la colección tiene configurado `Bearer {{token}}`, así que todas las peticiones de rutas envían el token sin configurarlo una a una. Las de autenticación y la de "sin token" usan *No Auth*.
- **Pruebas automáticas** con `pm.test` y `pm.expect` en cada petición: código de estado, campos de la respuesta (por ejemplo, que no se devuelve la contraseña, que la ruta tiene 7 puntos y 3 sectores) y mensajes de error en español.
- Se exportó en formato **Collection v2.1** para que cualquier compañero pueda importarla en Postman o ejecutarla con Newman.

4.8.3 Por qué

- **Postman** permite probar la API sin tener el frontend terminado y sirve como documentación viva de los endpoints para el resto del equipo.
- **Pruebas automáticas en la colección:** en lugar de mirar cada respuesta a mano, se ve al momento si algo se ha roto después de un cambio.
- **Variables y scripts:** evitan copiar y pegar el token a mano, que es la causa más habitual de errores al probar APIs con autenticación.

4.8.4 Problemas encontrados

- Al ejecutar la colección dos veces, el registro fallaba con 409 porque el email ya existía. Se resolvió generando un email distinto en cada ejecución con un *pre-request script*.

4.9 PS-18 · Entidad y tabla usuario en MySQL

Autor: Adam (integrado en el repositorio por Asier García) · **Fecha:** 21/10/2026 · **Issue:** PS-18

4.9.1 Qué se ha hecho

Se ha creado un backend en Spring Boot conectado a MySQL y, dentro de él, la entidad `Usuario`, que Hibernate convierte en la tabla `usuario`. Esta tabla guarda a los corredores de la aplicación y es la base para el registro y el inicio de sesión (PS-19).

4.9.2 Cómo se ha hecho

- **Proyecto Spring Boot 3.5 con Java 17**, con las dependencias `spring-boot-starter-data-jpa` (JPA/Hibernate) y `mysql-connector-j` (driver de MySQL).
- **Conexión** en `application.properties`. La URL, el usuario y la contraseña se leen de las variables `MYSQL_URL`, `MYSQL_USER` y `MYSQL_PASSWORD`; si no existen se usan valores de desarrollo (`localhost:3306`, `root`, sin contraseña). La opción `createDatabaseIfNotExist=true` crea la base de datos `ruta_fantasma` si no existe.
- **Entidad `Usuario`** (`modelo/Usuario.java`) con las anotaciones de JPA:
  - `@Entity` y `@Table(name = "usuario")` para indicar la tabla.
  - `@Id` y `@GeneratedValue(strategy = IDENTITY)` para que el `id` sea autoincremental en MySQL.
  - `@Column(unique = true)` en el email, para que no se repita.
  - Un método `@PrePersist` que rellena `fecha_registro` automáticamente al insertar.
- **Repositorio `UsuarioRepository`**, que hereda de `JpaRepository` y añade `findByEmailIgnoreCase` y `existsByEmailIgnoreCase`. Spring genera el SQL a partir del nombre del método.
- **Creación de la tabla:** con `spring.jpa.hibernate.ddl-auto=update` Hibernate crea la tabla al arrancar. Además se incluye el script `01-usuario.sql` para crearla a mano y como documentación.
- **Pruebas** con `@DataJpaTest` sobre una base de datos H2 en memoria en modo MySQL: se comprueba que se genera el `id` y la fecha, que la búsqueda por email no distingue mayúsculas y que no se pueden guardar dos usuarios con el mismo email. Pasan las 3.

4.9.3 Por qué

- **MySQL y JPA:** son las tecnologías vistas en clase para bases de datos relacionales; con JPA se trabaja con objetos Java y no hace falta escribir SQL a mano para lo básico.
- **Guardar `password_hash` y no la contraseña:** si alguien accediera a la base de datos no podría leer las contraseñas. El cifrado (BCrypt) se añade en la tarea PS-19.
- **Email único:** el email es el dato con el que se inicia sesión, así que no puede haber dos cuentas con el mismo.
- **Credenciales en variables de entorno:** así la contraseña de MySQL nunca se sube al repositorio.
- **Pruebas con H2:** permiten comprobar la tabla sin tener MySQL instalado.

4.9.4 Problemas encontrados

- Para poder probar sin MySQL se añadió H2 solo en el ámbito de pruebas (`scope test`), con su propio `application.properties` en `src/test/resources`.
- Esta tarea se subió primero como una carpeta aparte (`src/backend/PS-18-entidad-tabla-usuario/`) con un segundo proyecto Spring Boot, y se retiró. Para que exista un único backend, el código se integró en el proyecto `src/backend/backend-springboot/`, que pasa de MongoDB a MySQL. Se eliminaron el código anterior con MongoDB y el esqueleto de Node/Express, y se actualizó el stack en `CLAUDE.md`.

4.10 PS-23 · Tablas ruta, carrera, punto y sector

Autor: Adam (integrado en el repositorio por Asier García) · **Fecha:** 21/10/2026 · **Issue:** PS-23

4.10.1 Qué se ha hecho

Se han creado en MySQL las tablas que guardan los datos de las carreras: `ruta` (el recorrido), `punto` (sus puntos GPS), `sector` (los tramos en los que se divide, como en la Fórmula 1) y `carrera` (cada vez que alguien corre una ruta). Junto con la tabla `usuario` de PS-18 forman el modelo de datos completo de la aplicación.

4.10.2 Cómo se ha hecho

- **Una entidad JPA por tabla** (`Ruta`, `Punto`, `Sector`, `Carrera`) en el paquete `modelo`.
- **Relaciones:**
  - Un usuario tiene muchas rutas y muchas carreras (`@ManyToOne` hacia `Usuario`).
  - Una ruta tiene muchos puntos, muchos sectores y muchas carreras. En `Ruta`, los puntos y sectores son `@OneToMany` con `cascade = ALL` y `orphanRemoval = true`, así que se guardan y se borran junto con la ruta. Con `@OrderBy` siempre se leen en orden.
  - Los métodos `anadirPunto` y `anadirSector` de `Ruta` mantienen sincronizados los dos lados de la relación y numeran los puntos automáticamente.
- **Integridad de los datos:**
  - Claves foráneas con nombre (`fk_punto_ruta`, `fk_carrera_usuario`...).
  - Restricciones únicas: no puede haber dos puntos con el mismo `orden` ni dos sectores con el mismo `numero` en una ruta.
  - `ON DELETE CASCADE` (con `@OnDelete` de Hibernate): si se borra una ruta directamente en MySQL, también se borran sus puntos, sectores y carreras.
- **Carga diferida (`FetchType.LAZY`)** en todas las relaciones `@ManyToOne`, para no traer datos de la base de datos que no se van a usar.
- **Repositorios** `RutaRepository` (rutas de un usuario, de la más nueva a la más antigua) y `CarreraRepository` (carreras de una ruta y la más rápida, que es la "sombra").
- **Scripts SQL** (`01-usuario.sql` y `02-ruta-punto-sector-carrera.sql`) equivalentes a lo que genera Hibernate, para crear las tablas a mano y como documentación.

- **Pruebas** con `@DataJpaTest` (5 nuevas, 14 en total): guardado en cascada y en orden, borrado en cascada, cálculo de la sombra, rutas por usuario y la restricción de orden único.

4.10.3 Por qué

- **Tablas separadas para puntos y sectores** en lugar de guardarlos como texto dentro de la ruta: en una base de datos relacional cada dato va en su sitio (primera forma normal) y así se pueden consultar y validar.
- **El sector guarda el orden de los puntos de inicio y fin** en lugar de coordenadas: los puntos ya existen en la tabla `punto` y así no se repiten datos.
- **Tabla `carrera`** separada de la ruta: una misma ruta se corre muchas veces y hay que comparar cada intento con el mejor (la sombra), que es la idea central de Ruta Fantasma.
- **El tiempo en segundos (`BIGINT`)**: es fácil de sumar y comparar, y el frontend ya trabaja con `tiempoSegundos`.

4.10.4 Problemas encontrados

- Por defecto, Hibernate crea las claves foráneas sin `ON DELETE CASCADE`. Se añadió `@OnDelete(action = CASCADE)` para que la tabla generada sea igual que la del script SQL.
- Al integrar la tarea en el repositorio se añadió `CarreraRepository` con los dos métodos que usan las pruebas (`findFirstByRutaIdOrderByTiempoTotalSegundosAsc` para la sombra y `findByRutaIdOrderByFechaDesc` para el historial).

4.11 Carpeta de diseño para los mockups

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** —

4.11.1 Qué se ha hecho

Se ha creado en el repositorio la carpeta `docs/diseno/`, destinada a guardar los mockups de las pantallas de Ruta Fantasma antes de programarlas. Incluye una guía que explica cómo exportar y nombrar los diseños, y una tabla para seguir el estado de cada pantalla.

4.11.2 Cómo se ha hecho

- Se creó `docs/diseno/mockups/`, donde se subirán las imágenes PNG exportadas desde Figma.
- Se añadió `docs/diseno/README.md` con el espacio para el enlace al archivo de Figma, los pasos para exportar un mockup, la regla de nombres (`zona-pantalla.png`, con las mismas zonas que el frontend: `reloj`, `app` y `web`) y una tabla con las cinco pantallas previstas para el primer diseño y su estado (Pendiente, En revisión o Validado).
- Se actualizó el árbol del repositorio y la tabla de decisiones de `CLAUDE.md`.

4.11.3 Por qué

Figma guarda los diseños en la nube, por lo que no quedarían registrados en el repositorio. Subir las imágenes exportadas permite que todo el equipo y el profesor consulten los diseños directamente desde GitHub y que quede constancia de cómo evolucionan. Usar los mismos nombres de zona que el frontend facilita relacionar cada mockup con el código de su pantalla, y la columna de estado deja claro qué diseños ha validado ya el Product Owner.

4.11.4 Problemas encontrados

Se descartó el nombre `diseño` para la carpeta porque las tildes en los nombres de carpetas pueden dar problemas en algunos sistemas y herramientas; se usó `diseno`.

4.12 PS-26 · Pantalla con la lista de rutas

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-26

4.12.1 Qué se ha hecho

Se ha creado la pantalla “Mis rutas” (`/app`), que muestra las carreras guardadas en el backend. De cada ruta enseña el nombre, la distancia en kilómetros, el tiempo de la “sombra” (lo que se tardó en esa carrera) y la fecha. La pantalla contempla los cuatro estados posibles: cargando, error, lista vacía y lista con datos.

4.12.2 Cómo se ha hecho

- **Modelos** (`core/modelos/ruta.ts`). Las interfaces `Ruta` y `PuntoGps` copian exactamente los DTO del backend Spring Boot de Adam (`RutaResponse` y `PuntoGps`), para que los datos encajen sin conversiones.
- **Servicio `Rutas`** (`core/servicios/rutas.ts`). Usa el servicio `Api` de la tarea PS-21 para llamar a `GET /api/rutas` y `GET /api/rutas/{id}`.
- **Utilidades** (`compartido/utilidades/rutas.ts`). Son funciones puras, reutilizables por otras zonas (por ejemplo, el reloj):
  - `distanciaKm`: suma la distancia entre cada par de puntos GPS con la fórmula del semiverseno (*haversine*), que tiene en cuenta la curvatura de la Tierra.
  - `duracionSegundos`: calcula el tiempo entre el primer y el último punto.
  - Funciones de formato en español: `27:41`, `5,2 km` y `18 oct 2026`.
- **Componente `ListaRutas`** (`zonas/app/lista-rutas/`). Guarda el estado de la pantalla en *signals* y usa un `computed` que ordena las rutas de la más reciente a la más antigua y prepara los textos.
  - Mientras carga, muestra tres tarjetas “esqueleto” que laten. La animación se desactiva si el usuario tiene activada la opción de reducir el movimiento.
  - Si falla, muestra el mensaje del interceptor de errores y un botón **Reintentar**.
- **Diseño.** Las tarjetas siguen el mockup “Elegir ruta” de Daniel (`docs/diseno/mockups/`): nombre en negrita, “sombra mm:ss” debajo y los kilómetros a la derecha.
- **Estructura de la zona `/app`.** `AppInicio` pasa a ser el marco de la zona (cabecera con la sesión y menú inferior) y contiene un `router-outlet`. La lista es su ruta hija, así las próximas pantallas de la app reutilizarán ese marco sin repetirlo.

- **Pruebas.**
  - **Pruebas unitarias:** se añadieron 12 nuevas (utilidades, servicio y pantalla en sus cuatro estados, incluido el botón Reintentar). En total pasan 47.
  - **Prueba en el navegador:** se usó una API falsa con el mismo formato que el backend, a tamaño de móvil. Se comprobaron la lista con tres rutas, el error con reintento y la lista vacía, y que cada petición lleva el token `Authorization: Bearer` de la tarea PS-20.

4.12.3 Por qué

- **Calcular la distancia y el tiempo en el frontend:** el backend solo guarda los puntos GPS, así que no hace falta cambiar la API para mostrar estos datos.
- **Funciones de cálculo en `compartido/`:** el reloj también necesitará calcular distancias y tiempos durante la carrera, y así no se duplica el código.
- **Mostrar siempre un estado claro:** nunca aparece una pantalla en blanco. El usuario sabe en todo momento si está cargando, si no hay datos o si ha fallado algo, y puede reintentarlo.

4.12.4 Problemas encontrados

- Al principio un test no compilaba porque se pasaba un valor de tipo unknown a `flush()` de `HttpTestingController`. Se corrigió tipando el parámetro como `Ruta[] | null`.
- Como el backend no puede ejecutarse en este equipo (no tiene Java ni MongoDB), se usó una API falsa en Node.js que devuelve rutas con el mismo formato que `RutaResponse`.

4.13 PS-20 · Pantallas de registro e inicio de sesión

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-20

4.13.1 Qué se ha hecho

Se han creado las pantallas de inicio de sesión (`/app/login`) y de registro (`/app/registro`) en la zona de la app móvil, junto con el servicio que gestiona la sesión del usuario. La cabecera de la app muestra ahora “Hola, ” y un botón **Salir** cuando hay sesión iniciada, o un enlace **Entrar** cuando no la hay.

4.13.2 Cómo se ha hecho

- **Formularios reactivos** (`ReactiveFormsModule`) con validaciones en el propio navegador:
  - **Login:** email obligatorio y con formato válido, y contraseña obligatoria.
  - **Registro:** nombre obligatorio (máximo 40 caracteres), email válido, contraseña de al menos 6 caracteres y confirmación. Un validador propio, `passwordsIguales`, comprueba que las dos contraseñas coinciden.
  - Los mensajes de error aparecen debajo de cada campo cuando el usuario ya lo ha tocado. Si se pulsa el botón con errores, se marcan todos los campos.

- **Servicio `Auth`** (`core/servicios/auth.ts`). Usa el servicio `Api` de la tarea PS-21 para llamar a `POST /api/auth/login` y `POST /api/auth/registro`. Guarda la sesión (token y usuario) en *signals* y en `localStorage`, para que siga abierta al recargar la página. Si `localStorage` está bloqueado o contiene datos corruptos, la aplicación sigue funcionando.
- **Interceptor `authInterceptor`.** Añade la cabecera `Authorization: Bearer <token>` a las peticiones que van a nuestra API, y solo a esas, para no enviar el token a otros servidores.
- **Guard `authGuard`** (`core/guardias/`). Queda preparado para proteger pantallas: si no hay sesión, redirige a `/app/login?volver=<página>` y, al entrar, el login devuelve al usuario a esa página.
- **Estados de la pantalla.** Mientras se espera al servidor, el botón se desactiva y cambia su texto ("Entrando…"). Los errores del servidor (credenciales incorrectas, email ya registrado, sin conexión) se muestran en un recuadro rojo con el mensaje en español que proporciona el interceptor de errores.
- **Estilos compartidos.** Como las dos pantallas comparten diseño, sus estilos están en `compartido/estilos/formulario.css` y cada componente los incluye con `styleUrls`.
- **Pruebas.**
  - **Pruebas unitarias:** se añadieron 19, que cubren el servicio, el interceptor, el guard y las dos pantallas: validaciones, envío correcto, que la confirmación no se envía al servidor y la gestión de errores 401 y 409. En total pasan 35.
  - **Prueba en el navegador:** se usó una API falsa que imita el backend. Se probó el formulario vacío, la contraseña incorrecta, el login correcto, contraseñas que no coinciden en el registro y el registro correcto.

4.13.3 Contrato con el backend

El backend todavía no tiene autenticación. El frontend espera estos dos endpoints, que habrá que implementar en Spring Boot:

Método y ruta	Cuerpo	Respuesta correcta	Errores
`POST /api/auth/login`	`{ email, password }`	`200 { token, usuario: { id, nombre, email } }`	`401` si los datos no son correctos
`POST /api/auth/registro`	`{ nombre, email, password }`	`201 { token, usuario: { id, nombre, email } }`	`409 { error }` si el email ya existe; `400` en las validaciones

Además, la configuración CORS del backend debe permitir la cabecera `Authorization`.

4.13.4 Por qué

- **Sesión guardada en el servicio, con *signals*:** cualquier pantalla puede saber al momento quién ha entrado, y la cabecera se actualiza sola al entrar o salir.

- **Validar en el navegador antes de enviar:** el usuario recibe respuesta inmediata y se evitan peticiones inútiles al servidor. El backend seguirá validando igualmente por seguridad.
- **El guard no se ha aplicado todavía a ninguna ruta.** Mientras el backend no tenga login, proteger la app impediría usarla. Cuando exista, basta con añadir `canActivate: [authGuard]` a las rutas privadas.

4.13.5 Problemas encontrados

- No se puede arrancar el backend en este equipo (no tiene Java ni MongoDB), y además todavía no tiene endpoints de autenticación. Se resolvió con pruebas que simulan el servidor (`HttpTestingController`) y, para las pruebas en el navegador, con una API falsa que sigue el contrato descrito arriba.
- Al añadir el servicio `Auth` a la pantalla principal de la app, sus pruebas necesitaban `HttpClient`. Se añadieron `provideHttpClient()` y `provideHttpClientTesting()` a esas pruebas.

4.14 PS-21 · Servicio HTTP base de Angular

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-21

4.14.1 Qué se ha hecho

Se ha creado la capa que comunica el frontend con el backend: un servicio HTTP base (`Api`) que usarán todos los demás servicios, un interceptor que traduce los errores del servidor a mensajes en español y la configuración de la dirección de la API por entornos.

4.14.2 Cómo se ha hecho

- **Entornos.** Con `ng generate environments` se crearon `src/environments/environment.t` (producción) y `environment.development.ts` (desarrollo). Ambos guardan `apiUrl`, que de momento apunta al backend Spring Boot local (`http://localhost:8080/api`). Angular sustituye un archivo por otro automáticamente según el tipo de compilación.
- **Servicio `Api`** (`core/servicios/api.ts`). Envuelve el `HttpClient` de Angular con cuatro métodos genéricos (`get`, `post`, `put` y `delete`). Recibe solo la ruta relativa (por ejemplo `'rutas'`) y construye la URL completa. Se usa el nuevo decorador `@Service()` de Angular 22, que registra el servicio automáticamente en la inyección de dependencias.
- **Interceptor de errores** (`core/interceptores/errores-interceptor.ts`). Es un interceptor funcional que se ejecuta en todas las peticiones. Cuando una falla, convierte el `HttpErrorResponse` en un objeto `ErrorApi` (`core/modelos/error-api.ts`) con el código, un mensaje en español y, si los hay, los errores de validación por campo. Está adaptado a las respuestas reales del backend de Adam: `{ error: '...' }` en los errores generales y `{ campo: 'mensaje' }` en las validaciones (400).
- **Registro.** En `app.config.ts` se añadió `provideHttpClient(withFetch(), withInterceptors([erroresInterceptor]))`.
- **Pruebas.** Con `HttpTestingController` se simulan las respuestas del servidor sin necesidad de tener el backend arrancado. Se comprueba que se construyen

bien las URL y los parámetros, que cada método usa el verbo HTTP correcto y que los errores 0 (sin conexión), 400, 404 y 500 se traducen correctamente. Se añadieron 6 pruebas y todas pasan.

4.14.3 Por qué

- **Un único punto de acceso al backend:** si cambia la dirección del servidor, se cambia en un solo archivo, y ningún componente necesita conocer `HttpClient`.
- **Errores centralizados:** las pantallas solo tienen que mostrar `error.mensaje`, sin repetir en cada una la lógica para interpretar los códigos HTTP. El usuario recibe siempre mensajes comprensibles en español.
- **`withFetch()`:** hace que Angular use la API `fetch` del navegador, que es la opción recomendada actualmente.

4.14.4 Problemas encontrados

- Angular 22 genera los servicios con el decorador `@Service()` en lugar del clásico `@Injectable({ providedIn: 'root' })`. Se consultó su definición para confirmar que el servicio se registra automáticamente en toda la aplicación.
- El backend no puede arrancarse en este equipo porque no tiene Java ni MongoDB. Por eso las pruebas simulan el servidor con `HttpTestingController`.

4.15 Flujo de trabajo con ramas y Pull Requests

Autor: Asier García · **Fecha:** 26/10/2026 · **Issue:** —

4.15.1 Qué se ha hecho

Se ha definido y documentado el flujo de trabajo en GitHub que exige el enunciado del proyecto (sección 5). Cada historia se desarrolla en su propia rama, nadie sube cambios directamente a `main`, y todo se integra mediante una Pull Request que revisa y aprueba otro desarrollador. Este cambio es el primero que se ha subido siguiendo el nuevo flujo.

4.15.2 Cómo se ha hecho

- **Reglas comunes en `CLAUDE.md`.** Se añadió una sección “Flujo de Git” que el asistente de cada desarrollador lee al abrir el proyecto, de modo que los tres trabajan igual sin necesidad de recordarlo cada vez:
  - **Ramas:** una por historia, con el nombre `PS-XX-descripcion-corta` (clave de Jira y 2–4 palabras), creada siempre desde `main` actualizado.
  - **Commits:** pequeños y frecuentes, con la clave de Jira en el mensaje (`PS-XX: descripción`).
  - **Integración:** antes de subir, se integra en la rama lo último de `main` (`git fetch` + `git merge origin/main`), de forma que el historial muestra que se incorpora el trabajo de los compañeros.
  - **Pull Request:** se sube solo la rama y se abre la Pull Request hacia `main`. Nunca se hace el merge de la PR propia: la aprueba otro DT y se fusiona en GitHub con *merge commit*, para que la rama quede visible en el historial.
- **Comandos del equipo.**

  - `/empezar PS-XX`: actualiza `main`, resume el estado del proyecto, avisa de las ramas abiertas de los compañeros y crea (o retoma) la rama de la tarea.
  - `/terminar`: escribe la entrada de memoria, ejecuta el build y los tests, hace commits por pasos, integra `main`, sube la rama y prepara la Pull Request.
  - `/revisar PS-XX` (nuevo): descarga la rama de un compañero, la compila, la prueba y redacta una revisión lista para pegar en GitHub, sin modificar su trabajo.
- **Plantilla de Pull Request** (`.github/pull_request_template.md`). Incluye la clave de Jira, cómo probar el cambio y la Definición de Hecho del enunciado, incluida la accesibilidad básica.
- **README.** La guía diaria del equipo explica ahora el ciclo completo: `/empezar` → trabajo en la rama → `/terminar` → revisión de otro DT → fusión.

4.15.3 Por qué

- **Lo exige el enunciado:** "nadie sube cambios directamente a `main`", "cada historia se desarrolla en su propia rama" y cada DT debe tener commits y Pull Requests propios en todos los sprints. Además, la Definición de Hecho exige que el código esté integrado en `main` mediante una Pull Request revisada.
- **Deja constancia del trabajo:** las ramas y las Pull Requests muestran quién ha hecho cada historia, quién la ha revisado y cuándo se ha integrado, lo que se usará en el apartado "Trabajo en GitHub" de la memoria final.
- **Revisión entre compañeros:** las revisiones permiten detectar errores antes de que lleguen a `main` y que todo el equipo conozca el código de los demás. Esto también lo pide el enunciado para el código generado con IA.

4.15.4 Problemas encontrados

- Las tareas DT2, PS-20, PS-21 y PS-26 se subieron directamente a `main` antes de conocer esta norma. No se ha reescrito ese historial, porque exigiría `git push --force` sobre la rama común. A partir de este cambio, todo entra por Pull Request.
- Para que GitHub impida de verdad subir a `main` sin revisión, el propietario del repositorio debe activar una regla de protección de la rama (*Require a pull request before merging*, con una aprobación).

5 Sprint 2

Fechas: *(inicio — fin)*

Objetivo del sprint: *(una frase)*

5.1 Sprint Planning

Historia / tarea	Responsable	Estimación
(HU-…)	*(nombre)*	*(pts)*

5.2 Trabajo realizado

5.3 PS-32 · Filtro de GPS: precisión y saltos imposibles

Autor: Asier García · **Fecha:** 28/10/2026 · **Issue:** PS-32

5.3.1 Qué se ha hecho

Se ha añadido un filtro que descarta las lecturas del GPS que no sirven para medir una carrera: las que tienen mala precisión, las que implicarían un salto imposible y las repetidas. El servicio `Gps` (PS-28) ofrece ahora, además de la posición en bruto, la última **posición válida**, que es la que usarán la grabación de rutas y la carrera. La pantalla GPS muestra cuántas lecturas se han descartado y por qué.

5.3.2 Cómo se ha hecho

- **Servicio `FiltroGps`** (`core/servicios/filtro-gps.ts`). Cada lectura pasa por tres comprobaciones, en este orden:
  1. **Precisión:** si el GPS indica un margen de error mayor de **20 m**, se descarta. Es el mismo umbral que la pantalla GPS de la PS-28 ya marcaba en verde.
  2. **Lectura repetida:** si llega con la misma hora que la última buena, o con una anterior, se descarta. El navegador a veces repite una posición guardada.
  3. **Salto imposible:** con el servicio `Calculo` de la PS-31 se calcula la distancia (Haversine) desde la última lectura aceptada y la velocidad necesaria para recorrerla. Si supera los **12 m/s (43 km/h)**, se descarta. Ni el récord del mundo de 100 m llega a esa velocidad media.
- **Referencia siempre a la última lectura buena.** Así, si se cuela una lectura mala, el filtro se corrige solo: el tiempo transcurrido desde ella va creciendo y las lecturas buenas siguientes vuelven a ser creíbles.
- **Integración en `Gps`** sin cambiar lo que ya existía:
  - Se añadieron las señales `posicionValida`, `descartadas` y `ultimoDescarte`. La señal `posicion` (en bruto) sigue igual, así que la pantalla y las pruebas de la PS-28 no cambian.
  - Se usa la hora real de la lectura (`GeolocationPosition.timestamp`) y, si el navegador no la da, la hora actual.
  - Cada vez que se inicia el GPS, el filtro se reinicia.
- **Pantalla GPS:** muestra “Lecturas descartadas: N (la última por …)” en una zona `aria-live`, para que los lectores de pantalla también lo anuncien.
- **Pruebas.**
  - **Pruebas unitarias:** se añadieron 14 nuevas: 9 del filtro (precisión, límite exacto de 20 m, valores no numéricos, movimiento normal, salto, comparación con la última buena, lecturas repetidas, recuperación tras una lectura mala y reinicio), 4 de la integración en `Gps` y 1 de la pantalla. En total pasan 80.
  - **Prueba en el navegador:** se simuló el GPS con 6 lecturas. Las 2 malas (precisión de 45 m y un salto de 1,1 km en un segundo) se descartaron, mientras la pantalla seguía mostrando la lectura en bruto.

5.3.3 Por qué

- **El GPS de un móvil comete errores:** al salir de un portal, bajo los árboles o entre edificios puede dar posiciones a decenas de metros o "saltar" de golpe. Si esas lecturas se usaran, la distancia de la carrera y la comparación con la sombra serían falsas, y se avisaría en rojo sin motivo.
- **Filtrar en el frontend y no en el servidor:** las lecturas llegan al dispositivo varias veces por segundo durante la carrera. Hay que descartarlas en ese momento, antes de calcular distancias o avisar al corredor.
- **Mantener también la posición en bruto:** sirve para depurar y para que la pantalla GPS siga mostrando lo que ve el dispositivo.

5.3.4 Problemas encontrados

- La pantalla GPS y su servicio son de Daniel (PS-28). Para no modificar su trabajo, el filtro se añadió como señales nuevas y sus pruebas originales siguen pasando sin cambios.
- En el navegador el número de descartes aparecía pegado al motivo ("2(la última…)"), porque Angular elimina los espacios entre elementos. Se arregló con un margen en el CSS.

5.4 PS-31 · Servicio de cálculo: fórmula de Haversine y distancia

Autor: Asier García · **Fecha:** 28/10/2026 · **Issue:** PS-31

5.4.1 Qué se ha hecho

Se ha creado en el frontend el servicio `Calculo`, que reúne todos los cálculos de la carrera: la distancia entre dos puntos GPS con la fórmula de Haversine, la distancia total de un recorrido, la velocidad y el ritmo (minutos por kilómetro). A partir de ahora el filtro del GPS, la grabación de rutas y el reloj calculan con el mismo código.

5.4.2 Cómo se ha hecho

- **Fórmula de Haversine** (`core/servicios/calculo.ts`, función `haversineMetros`). Calcula la distancia sobre la superficie de una esfera del radio medio de la Tierra (6 371 km) a partir de la latitud y la longitud de los dos puntos:
  - `h = sen²(Δlat/2) + cos(lat1) · cos(lat2) · sen²(Δlon/2)`
  - `d = 2 · R · arcsen(√h)`

  Se limita √h a 1 para evitar errores de redondeo en puntos casi opuestos del planeta.
- **Servicio `Calculo`** con el decorador `@Service()` de Angular 22:
  - `distancia(a, b)`: metros entre dos puntos.
  - `distanciaTotal(puntos)`: suma de los tramos entre puntos consecutivos.
  - `velocidad(metros, segundos)`: en m/s, devuelve 0 si el tiempo no es positivo para no dividir entre 0.
  - `ritmo(metros, segundos)`: en segundos por kilómetro, devuelve `null` si aún no hay distancia.
- **Interfaz `Coordenada`** (`{ latitud, longitud }`). Gracias al tipado estructural de TypeScript, el servicio acepta tanto los `PuntoGps` de las rutas como las `PosicionGps` que llegan del GPS en vivo.

- **Una sola fórmula en toda la app.** La utilidad `distanciaKm` de la lista de rutas (PS-26) tenía su propia copia de Haversine; ahora usa `haversineMetros`. Además se añadió `formatearRitmo`, que muestra el ritmo como en el mockup del reloj (`5:17 /km`).
- **Pruebas.**
  - **Pruebas unitarias:** se añadieron 10 nuevas con casos de resultado conocido: un grado de latitud ≈ 111,2 km, Puerta del Sol → Plaza Mayor ≈ 360 m, la distancia es simétrica, media vuelta a la Tierra = π · R, la distancia total, la velocidad sin dividir entre 0 y el ritmo de 5,2 km en 27:28 ≈ 5:17 /km. En total pasan 66.
  - **Compilación:** la build de producción se genera sin errores.

5.4.3 Por qué

- **Haversine y no Pitágoras sobre las coordenadas:** un grado de longitud no mide lo mismo en Madrid que en el ecuador. Haversine tiene en cuenta la curvatura de la Tierra y para distancias de carrera el error es inferior al 0,5 %, más que suficiente frente al error del propio GPS.
- **Un servicio común:** la distancia y la velocidad aparecen en varias partes de la app (filtro del GPS, grabación, reloj, resumen). Si cada una calculara a su manera, los números podrían no coincidir entre pantallas.
- **Trabajar en metros y segundos:** son las unidades del GPS y de la API. El paso a km y al formato `m:ss` se hace solo al mostrar los datos.

5.4.4 Problemas encontrados

- Había dos implementaciones de Haversine: la de la lista de rutas y la nueva. Se unificaron para que solo exista una, y se comprobó que las pruebas de la lista de rutas siguen dando los mismos resultados.

6 Conclusiones

(Se completa al final del proyecto: resultado obtenido, qué se aprendió, qué se mejoraría.)